

# BalatroQt

基于 Qt6 / C++17 的卡牌游戏复刻与扩展

程序设计实习课程大作业报告

---

|      |                                   |
|------|-----------------------------------|
| 组号   | 125                               |
| 组员   | 肖楚童、朱韬宇、罗铭皓                       |
| 技术栈  | C++17、Qt 6 Widgets、QtOpenGL、CMake |
| 提交文件 | 125-作业报告.pdf                      |
| 日期   | 2026 年 6 月 30 日                   |

---

目录

|          |                       |          |
|----------|-----------------------|----------|
| <b>1</b> | <b>程序功能介绍</b>         | <b>2</b> |
| 1.1      | 项目定位 . . . . .        | 2        |
| 1.2      | 核心玩法 . . . . .        | 2        |
| 1.3      | 已实现功能 . . . . .       | 2        |
| <b>2</b> | <b>项目各模块与类设计细节</b>    | <b>3</b> |
| 2.1      | 总体架构 . . . . .        | 3        |
| 2.2      | 卡牌与小丑设计 . . . . .     | 3        |
| 2.3      | 游戏状态与事件驱动计分 . . . . . | 4        |
| 2.4      | 商店、牌组与赌注系统 . . . . .  | 4        |
| 2.5      | 界面与渲染设计 . . . . .     | 4        |
| 2.6      | 课程特色机制 . . . . .      | 4        |
| <b>3</b> | <b>小组成员分工情况</b>       | <b>5</b> |
| <b>4</b> | <b>项目总结与反思</b>        | <b>5</b> |
| 4.1      | 项目成果 . . . . .        | 5        |
| 4.2      | 开发困难 . . . . .        | 5        |
| 4.3      | 后续改进方向 . . . . .      | 6        |
| 4.4      | 结语 . . . . .          | 6        |

## 摘要

BalatroQt 是一个基于 Qt6/C++17 实现的桌面端卡牌肉鸽游戏项目，目标是在 C++ 环境中复刻并扩展《Balatro》的核心玩法。项目已经实现牌局循环、牌型识别、事件驱动计分、商店经济、补充包、优惠券、标签、赌注、Boss 盲注、收藏图鉴、演示模式等系统，并结合课程内容设计了队列牌组、栈牌组、运算符重载小丑、类模板小丑、迭代器增强和浅拷贝塔罗等原创机制。本文按课程作业报告要求，从程序功能、模块与类设计、小组分工以及项目总结四个方面说明项目成果。

**关键词：** Qt6；C++17；卡牌游戏；事件驱动计分；面向对象设计；图形界面

# 1 程序功能介绍

## 1.1 项目定位

BalatroQt 是一个单机桌面游戏程序，使用 Qt Widgets、QGraphicsScene 与 QtOpenGL 构建图形界面。项目以《Balatro》的主要规则为参照，复刻“选择盲注、出牌或弃牌、结算分数、进入商店、推进底注、挑战 Boss 盲注”的核心循环。与简单的纸牌演示不同，本项目同时处理牌局状态、卡牌效果、资源经济、界面动画、音效反馈和资源渲染，形成了一个可实际运行的中型 C++ 应用。

从课程目标看，本项目还承担了“将程序设计概念转化为游戏机制”的任务。队列牌组和栈牌组分别对应先进先出与后进先出的数据结构思想；运算符重载、类模板、迭代器、浅拷贝等概念被设计为可触发、可展示、可参与计分的卡牌效果，使课程知识点不只停留在语法层面，而能在游戏过程中被直观体验。

## 1.2 核心玩法

玩家每回合从牌组抽取手牌，在有限的出牌次数和弃牌次数内获得足够分数。每次出牌最多选择五张牌，系统根据点数、花色和特殊效果判断牌型，再计算基础筹码和基础倍率。最终得分以“筹码 × 倍率”的方式累积，并受到增强牌、版本、蜡封、小丑牌、Boss 盲注、赌注贴纸和牌组规则影响。

通过当前盲注后，玩家进入商店阶段。商店提供小丑牌、塔罗牌、星球牌、幻灵牌、补充包和优惠券等资源。玩家在有限金币下选择构筑方向，并为后续盲注做准备。通过 Boss 盲注后，游戏进入下一底注，目标分数和限制条件继续提升。直到通关 8 底注 Boss 便视为游戏胜利，可以选择开始新的一局或者继续挑战无尽模式。

## 1.3 已实现功能

项目当前功能可以概括为以下几类：

- **牌局与计分：**支持 13 类牌型识别，包含高牌、对子、两对、三条、顺子、同花、葫芦、四条、同花顺、皇家同花顺、五条、同花葫芦、同花五条等；计分过程采用事件流组织，便于 UI 逐项播放动画。
- **卡牌效果：**支持普通扑克牌的增强、版本、蜡封、debuff 与永久加筹；支持小丑牌、塔罗牌、星球牌和幻灵牌等多类效果。
- **商店经济：**实现商品刷新、购买、出售、重掷、折扣、优惠券、补充包和标签奖励等机制。
- **盲注与赌注：**实现小盲、大盲、Boss 盲注、底注推进以及不同赌注等级的规则限制。
- **牌组系统：**复刻基础牌组选择逻辑，并加入队列牌组和栈牌组等课程特色牌组。

- **贴纸系统**：实现永恒、租用、易腐等原版贴纸效果，支持在商店、收藏图鉴、比赛信息和牌局结算中正确显示与生效。
- **收藏与演示**：提供收藏图鉴界面、卡牌悬浮信息、演示模式和课堂路演所需的固定流程。
- **视听反馈**：使用像素风贴图、卡牌阴影、拖拽动画、开包动画、商店动画、计分动画、GLSL 着色器和音效反馈提升交互表现。

在上述功能中，牌局主循环是各系统联动的核心，其主要流程如下表所示。

| 流程环节  | 功能说明                                 |
|-------|--------------------------------------|
| 盲注选择  | 展示小盲、大盲和 Boss 盲注，支持跳过奖励与标签触发。        |
| 出牌与弃牌 | 根据手牌、牌组约束和 Boss 规则决定可选牌，并更新剩余次数。     |
| 计分结算  | 生成基础分与 ScoreEvent 序列，驱动筹码、倍率和小丑触发动画。 |
| 商店阶段  | 处理商品、优惠券、补充包、金币、重掷和槽位变化。             |
| 底注推进  | 通过 Boss 后进入下一 Ante，目标分数和赌注限制继续提高。    |

## 2 项目各模块与类设计细节

### 2.1 总体架构

项目采用分层结构，依赖方向为 ui -> game -> card -> utils。其中 game 层维护纯游戏规则，ui 层只通过接口和信号响应状态变化，避免界面直接篡改游戏状态。这样的结构使得复杂规则可以集中在 GameState、HandEvaluator 和 Shop 等核心类中维护，也便于后续继续扩展小丑、牌组、赌注和商店规则。

| 模块        | 代表类或文件                                                   | 主要职责                       |
|-----------|----------------------------------------------------------|----------------------------|
| src/card  | CardData, Joker, CardItem                                | 定义卡牌、小丑和消耗牌的数据结构与图形对象。     |
| src/game  | GameState, HandEvaluator, Shop, BossBlind                | 维护牌局状态、计分、商店、盲注、赌注和牌组规则。   |
| src/ui    | MainWindow, ShopWidget, PackOpenWidget, DeckSelectWidget | 负责界面、动画、悬浮信息、场景切换和用户交互。    |
| src/utils | constants.h, ShaderEffects, GpuEffectsRenderer           | 提供常量配置、CPU/GPU 图像效果和着色器工具。 |
| src/audio | AudioManager                                             | 统一管理背景音乐与音效播放。             |

### 2.2 卡牌与小丑设计

CardData 是扑克牌的值类型，封装花色、点数、增强、版本、蜡封、debuff 标记和永久加筹等信息。每张牌拥有进程内唯一的 uid，动画和移动逻辑通过 uid 追踪卡牌，而不是依赖数组下标。这可以避免洗牌、复制、移动或销毁卡牌时出现对象错位。

Joker 使用类型枚举、触发时机和回调函数组织效果。每张小丑牌由 JokerType、TriggerTiming

和效果函数共同描述。新增小丑时只需补充触发逻辑，而不必重写主计分流程。这种设计符合开闭原则，适合维护大量差异化效果。

`CardItem`、`JokerItem` 和 `ConsumableItem` 是对应的 `QGraphicsObject` 视图对象，负责绘制卡面、阴影、拖拽、悬停、翻面、缩放和触发动画。数据对象与显示对象分离，使规则计算和视觉表现可以分别维护。

## 2.3 游戏状态与事件驱动计分

`GameState` 是项目的单一事实源，保存手牌、小丑、金币、分数、盲注进度、消耗品、优惠券、赌注、标签和本局统计数据。界面层通过 `handChanged`、`scoreChanged`、`jokersChanged`、`shopChanged` 等信号刷新显示，而不是直接修改内部状态。

牌型识别和基础计分由 `HandEvaluator::evaluate` 完成。它不仅返回最终数值，还返回 `ScoreEvent` 列表。计分牌筹码、增强倍率、版本加成、钢铁牌乘区、小丑加成、红蜡封重触发、玻璃牌破碎等效果都会成为独立事件。UI 逐项回放这些事件，从而把“算分逻辑”和“演出动画”解耦。

## 2.4 商店、牌组与赌注系统

`Shop` 管理商店货架、优惠券和补充包。优惠券不是简单的显示项，而会改变商店槽位、商品概率、折扣、重掷成本和特殊商品规则。补充包则负责生成并展示可选内容，和塔罗、星球、幻灵等消耗牌系统联动。

牌组规则集中在 `GameDeckType` 及其派生类中。`name()`、`description()` 等接口由派生类实现，出牌次数、弃牌次数、选牌窗口、是否允许排序等规则通过虚函数覆盖。原版牌组、队列牌组和栈牌组都通过同一套接口接入主流程，体现了继承与多态的使用。

赌注系统负责在不同难度下修改牌局规则，例如改变目标分数、应用贴纸、限制资源和影响盲注信息显示。该系统与牌组选择、比赛信息面板、计分旁赌注图标和收藏图鉴都有交互，因此被集中纳入游戏状态层维护，减少 UI 层重复判断。

## 2.5 界面与渲染设计

`MainWindow` 是视图层调度中心，连接 `GameState` 信号与界面刷新，并管理对局、商店、开包、收藏、牌组选择、回合结算、启动页和选项菜单等场景。计分动画依赖 `QPropertyAnimation` 与 `QTimer` 协调，保证不同分数事件按顺序播放。

GPU 特效由 GLSL 片元着色器完成，覆盖金箔、全息、多彩、负片、火焰、漩涡、CRT 等效果。若 OpenGL 初始化失败，系统保留 CPU 回退路径，降低图形环境差异导致的风险。全屏 OpenGL 场景中容易出现原生弹窗黑屏问题，因此项目尽量采用场景内覆盖层实现菜单和提示框，避免破坏渲染上下文。

## 2.6 课程特色机制

项目把部分 C++ 程序设计概念设计为游戏内机制，使课程知识与玩法结合。

| 概念    | 游戏内实现                                 |
|-------|---------------------------------------|
| 队列    | 队列牌组按抽牌顺序排列手牌，只能从队首窗口选牌，体现先进先出约束。     |
| 栈     | 栈牌组要求出牌或弃牌包含最新抽入的栈顶牌，体现后进先出约束。        |
| 运算符重载 | 运算符重载小丑在计分事件流中交换筹码与倍率贡献，再按 UI 事件回放结算。 |
| 类模板   | 类模板小丑记录每底注首次打出的牌型，并将其视作一次模板实例化。       |
| 迭代器   | 迭代器增强使被打出的牌点数递增并循环，模型逻辑和翻面动画共享同一规则。   |
| 浅拷贝   | 浅拷贝塔罗使两张牌共享状态，一侧变化时同步点数、花色、增强、版本和蜡封。  |

此外，项目实现了两个原创辅助算法。占卜功能用于分数预测和胜率预览，在不写入真实状态的前提下模拟当前出牌得分；最佳出牌功能使用位掩码枚举合法出牌组合，并结合牌组约束、Boss 限制和顺序相关小丑寻找推荐出牌。

### 3 小组成员分工情况

| 成员  | 主要分工                                  |
|-----|---------------------------------------|
| 肖楚童 | 负责核心游戏逻辑、计分事件流、状态管理以及版本管理与 Git 工作流维护。 |
| 朱韬宇 | 负责界面动画、GLSL 着色器特效、路演演示模式和主要视觉反馈。      |
| 罗铭皓 | 负责牌组体系设计、继承与多态机制、程序设计概念卡和原创算法设计。      |

小组协作采用 Git 管理版本。项目开发过程中使用 `develop` 与 `main` 分支区分日常迭代和稳定版本，功能修复后再合并到主线。这一流程降低了多人协作中代码互相覆盖、功能回退和资源文件遗漏的风险。

### 4 项目总结与反思

#### 4.1 项目成果

BalatroQt 已经形成一个功能较完整的桌面游戏项目。它不仅实现了基础牌局循环，还覆盖商店、卡包、优惠券、标签、赌注、收藏、牌组选择、演示模式和视觉特效等系统。项目核心 C++/Header 代码约 3.4 万行，按整体项目口径约 3.8 万行；其中 `MainWindow` 和 `GameState` 承担了大量界面调度与状态管理工作。

工程设计上的主要成果，是使用单一事实源和事件驱动计分结构组织复杂规则。新增卡牌效果时，模型层生成明确的计分事件，界面层按事件播放动画。这样既保证最终得分可追踪，也让动画表现不必侵入规则计算。

#### 4.2 开发困难

项目最大的困难来自复杂系统之间的联动。小丑、版本、蜡封、Boss 盲注、赌注贴纸和商店优惠都会影响同一局游戏状态。如果缺少统一状态源和事件流，调试成本会迅速上升。后期通过 `GameState` 与 `ScoreEvent` 集中管理，项目可维护性有所提高。

第二个困难来自 UI 与底层渲染的耦合。Qt Widgets、QGraphicsScene 与 OpenGL 同时使用时，原生弹窗、全屏窗口和 GPU 后处理可能产生黑屏或重绘问题。项目改用场景内覆盖层实现菜单和提示框，说明图形界面开发不仅要实现控件功能，还要验证不同渲染路径的兼容性。

第三个困难是项目规模增长带来的维护压力。当前部分文件仍然较大，尤其是 `MainWindow`

和 `GameState`。虽然集中实现便于快速迭代，但长期维护时应继续拆分收藏界面、牌组选择、商店、计分动画和规则子模块。

### 4.3 后续改进方向

后续若继续开发，可以从以下方面改进：

- 为牌型识别、计分事件、赌注规则和牌组限制等功能补充自动化测试，减少风险引入。
- 优化动画等 UI 设计，校准 UI 比例、动画节奏、卡牌说明和触发时机，贴合原版游戏体验。
- 增加更加新颖有趣、贴合实际校园生活以及程设课程的游戏卡牌及相关逻辑，提高趣味性。
- 优化收藏界面、卡牌预览和翻页逻辑中的资源缓存，减少重复贴图合成和卡顿。

### 4.4 结语

本项目使小组完整经历了一次从规则建模、界面实现、动画组织到版本协作的综合开发过程。BalatroQt 的价值不只在于复刻一款游戏，也在于把 C++ 程序设计概念嵌入可交互系统中。通过该项目，我们进一步理解了面向对象设计、事件驱动架构、图形界面开发和团队版本管理在中型项目中的实际意义。